Quick Defense Sheet — 1 / 3

Purpose and users

The Barangay Minante 1 Integrated Management System is a server-rendered Flask application for resident records and permit, event, blotter, scheduling, notification and supporting service workflows. It is designed to centralize tracking and administrative records. No survey result or measured community-wide improvement is claimed.

Resident: submits/tracks own requests, evidence, schedules and program applications. **Staff:** assists residents, verifies, endorses and handles front-desk operations; cannot process own transactions. **Administrator:** controls accounts, catalogs, final decisions, signatures, reports and financial authority. Assignment does not add permissions.

Technology and database

Python + Flask + Flask-SQLAlchemy + SQLite; Jinja HTML, custom CSS and JavaScript. Werkzeug hashes passwords. python-docx creates editable documents; matplotlib creates chart images; custom Canvas code draws report charts. Flask-Login, Chart.js and Bootstrap are not used. ReportLab builds this documentation PDF only.

35 SQLite tables were inspected. Examples: users, resident_profiles, permit_applications, application_requirements, event_requests, event_reservations, payments, community_programs, program_applications and activity_logs. PK identifies a row; FK links related rows. Schedule related_type/related_id is a logical link, not a foreign key to three tables.

Main workflows

Account: Register → Pending → ID/address verification → Administrator approval → authorized login. ID/address changes restart relevant review; account approval does not automatically revert.

Permit: Configured service → evidence and fee snapshots → acknowledgment → Staff verification → endorsement → Admin decision → payment if needed → designated Admin signature → Ready for Pickup → identity-confirmed release → Completed.

Event: Category/venue/interval → serialized conflict check → reserve or offer private ten-minute recommendation → review/endorse → Admin decision. Adjacent intervals are allowed; overlaps at the same venue are blocked.

Blotter: Record complainant/respondent/incident → Filed → review/investigation → hearing-state → Resolved → Closed. Detailed resolution/officer editing is incomplete; an appointment is separate.

Quick Defense Sheet — 2 / 3

Supporting services

Walk-in: Staff selects Resident → captures permit/evidence → records Resident ownership and encoder → daily queue → Walk-in Transaction Acknowledgment → normal permit flow. The helper does not independently require every assisted profile to be active/approved.

Queue: Waiting → Now Serving → Serving → Completed. Daily number uniqueness and conditional stage changes protect tested cases. Stored priority_category does not implement automatic priority ranking.

Payment: Not Required for zero fee; otherwise Unpaid until Paid/Waived. Paid is the exact Cash fee snapshot. Only Admin can waive with a reason. Authorized Staff may collect but cannot record own payment. Configured fee is not actual collection.

Program: Draft Scholarship/Ayuda → Admin publication → dated applications → review/correction → Qualified → Approved/Waitlisted → Admin-confirmed completion. Per Resident/Household claim keys prevent one-time duplicate claims; Repeatable permits repeats. Human personnel decide eligibility.

Announcement: Draft → approved Scheduled/Published → Expired → retained history. Queries enforce time boundaries; backend access persists lifecycle transitions. No background scheduler.

Documents and reporting

Submission Acknowledgment = intake. Walk-in Acknowledgment = assisted intake. Payment Receipt = actual positive Paid record. Approved Permit = approved, payment-eligible and electronically signed document.

Reports use real SQLite aggregates, applicable filters and 17 report types. Lines show trends, bars compare categories, donuts show composition, stacked bars compare month/current-status composition. Empty records do not produce invented data. Administrator DOCX exports have native text/tables and chart pictures with editable source tables. Application PDF export is absent; this PDF is documentation.

Important code

`load_user_and_csrf()` loads identity and checks mutation tokens; `permission_required()` checks general grants; `create_requirement_snapshot()` preserves historical evidence rules; `reserve_event()` serializes venue reservation; `payment_eligible()` gates service progression; `sign_approved_permit()` validates designated signer; `release_walkin_permit()` records one eligible release; `build_report()` and `word_report()` generate aggregate reporting output.

Quick Defense Sheet — 3 / 3

Security and integrity

Password hashes, signed sessions, CSRF checks, role/ownership checks, private uploads, type/signature/size validation, hashed expiring reset tokens and explicit activity logs are implemented. Cookie Secure depends on HTTPS configuration. Upload signature checking is not malware scanning. Signature images are electronic signatures, not cryptographic digital signatures.

Transactions group related writes. Flush obtains IDs without committing; commit saves; rollback discards pending SQL. File staging handles upload cleanup separately. Unique payment/claim/queue keys and conditional verification/release updates protect specific duplicate/race cases. SQLite WAL and busy timeout help concurrency but do not remove the single-writer constraint.

Evidence to cite

Use Chapter 9 and documentation/acceptance-run.txt for the latest executed suite count; the earlier October 5 run passed 35 tests. Saved Edge browser evidence covers 160 pages across three roles and four widths with no recorded JavaScript errors or horizontal overflow. Those are synthetic scenario results, not production certification.

The 5,000-permit synthetic benchmark is post-change only: Permit list 12.83 ms; Permit analytics 24.48 ms; Word export approximately 1.50 seconds. No measured before/after speedup is claimed. The read-only SQLite integrity check returned ok with no foreign-key violations.

Honest limits

Email/SMS are simulated; no payment gateway/refunds; recovery email delivery is absent; appointment Rescheduled does not edit time; some event/schedule time handling is inconsistent with explicit Manila conversion elsewhere. Priority ordering, full referral/concern closure and household move-out are incomplete. Pre-approval correction portal is absent. Sensitivity labels do not add fine-grained Staff clearance. Installed Word, physical printing and live provider delivery remain unverified.

Maintenance and future work

Use the persistent SQLite configuration with one service instance. Run backup_sqlite.py for a consistent database copy and coordinate private-file backups; test restoration. The supplied Render Free path is ephemeral demonstration storage. Keep database, IDs, signatures, .env and secrets out of GitHub. Complete partial workflows/time handling before adding provider integrations or larger deployment features.

Defense principle: explain what the current code actually does, show the evidence, state the boundary, and identify future work separately.